

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Системы искусственного интеллекта
Лабораторная работа №1

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Королева Юлия Александровна

Содержание:

Задание.....	3
Алгоритмы.....	4
Исходный код.....	5
Реализация линейной регрессии:.....	5
Реализация логистической регрессии:.....	10
Применение линейной регрессии:.....	14
Применение логистической регрессии:.....	15
Оценки моделей.....	16
Вывод.....	17

Задание

Используя Jupiter Notebook, реализовать алгоритмы линейной и логистической регрессий, используя объектно-ориентированное программирование. Провести обучение модели на двух датасетах, вывести графики изменений соответствующих ошибок и метрик.

Алгоритмы

Псевдокоды для алгоритмов линейной (а) и логистической (б) регрессий:

а) $y_{\text{pred}} = X * w + b$
 $\text{error} = y_{\text{pred}} - y$
 $\text{grad_w} = (1/n) * X^T * \text{error}$
 $\text{grad_b} = (1/n) * \Sigma(\text{error})$
 $w = w - \alpha * \text{grad_w}$
 $b = b - \alpha * \text{grad_b}$

б) $\text{sigmoid}(z) = 1 / (1 + \exp(-z))$
 $z = X * w + b$
 $y_{\text{pred}} = \text{sigmoid}(z)$
 $\text{error} = y_{\text{pred}} - y$
 $\text{grad_w} = (1/n) * X^T * \text{error}$
 $\text{grad_b} = (1/n) * \Sigma(\text{error})$
 $w = w - \alpha * \text{grad_w}$
 $b = b - \alpha * \text{grad_b}$

Исходный код

Ссылка на исходный код: [SysAI_lab1.ipynb](#)

Далее на листингах 1-4 представлена реализация классов регрессий и их применение.

Листинг 1 - Реализация линейной регрессии

```
@dataclass
class TrainingHistory:
    losses: list[float]
    weights_history: list[np.ndarray]
    bias_history: list[float]
    train_loss_history: list[float]
    val_loss_history: list[float]
    train_mae_history: list[float]
    val_mae_history: list[float]
    train_rmse_history: list[float]
    val_rmse_history: list[float]

class LinearRegression:
    def __init__(self, learning_rate: float = 0.01, n_iterations: int = 10000, tolerance: float = 1e-6, fit_intercept: bool = True, verbose: bool = False):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.tolerance = tolerance
        self.fit_intercept = fit_intercept
        self.verbose = verbose

        self.weights = None
        self.bias = 0.0 if fit_intercept else None
        self.is_fitted = False
        self.history = None

    def compute_loss(self, y_true: np.ndarray, y_pred: np.ndarray) -> float:
        return np.mean((y_true - y_pred) ** 2)

    def compute_mae(self, y_true: np.ndarray, y_pred: np.ndarray) -> float:
        return np.mean(np.abs(y_true - y_pred))

    def compute_rmse(self, y_true: np.ndarray, y_pred: np.ndarray) -> float:
```

```

        return np.sqrt(self.compute_loss(y_true, y_pred))

    def gradient_descent(self, X: np.ndarray, y: np.ndarray, X_val:
np.ndarray = None, y_val: np.ndarray = None):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        if self.fit_intercept:
            self.bias = 0.0

        losses = []
        weights_history = []
        bias_history = []
        train_loss_history = []
        val_loss_history = []
        train_mae_history = []
        val_mae_history = []
        train_rmse_history = []
        val_rmse_history = []

        for iteration in range(self.n_iterations):
            y_pred = np.dot(X, self.weights)
            if self.fit_intercept:
                y_pred += self.bias

            train_loss = self.compute_loss(y, y_pred)
            train_mae = self.compute_mae(y, y_pred)
            train_rmse = self.compute_rmse(y, y_pred)

            train_loss_history.append(train_loss)
            train_mae_history.append(train_mae)
            train_rmse_history.append(train_rmse)
            losses.append(train_loss)
            weights_history.append(self.weights.copy())
            if self.fit_intercept:
                bias_history.append(self.bias)

            val_loss = val_mae = val_rmse = None
            if X_val is not None and y_val is not None:
                y_val_pred = np.dot(X_val, self.weights)
                if self.fit_intercept:
                    y_val_pred += self.bias

                val_loss = self.compute_loss(y_val, y_val_pred)
                val_mae = self.compute_mae(y_val, y_val_pred)
                val_rmse = self.compute_rmse(y_val, y_val_pred)

```

```

        val_loss_history.append(val_loss)
        val_mae_history.append(val_mae)
        val_rmse_history.append(val_rmse)

    error = y_pred - y
    dw = (2 / n_samples) * np.dot(X.T, error)

    self.weights -= self.learning_rate * dw

    if self.fit_intercept:
        db = (2 / n_samples) * np.sum(error)
        self.bias -= self.learning_rate * db

    if iteration > 0 and abs(losses[-2] - losses[-1]) <
self.tolerance:
        if self.verbose:
            print(f"Converged at iteration {iteration}")
            break

    if self.verbose and iteration % 100 == 0:
        msg = f"Iteration {iteration}, Train Loss:
{train_loss:.6f}"
        if val_loss is not None:
            msg += f", Val Loss: {val_loss:.6f}"
        print(msg)

self.history = TrainingHistory(
    losses=losses,
    weights_history=weights_history,
    bias_history=bias_history,
    train_loss_history=train_loss_history,
    val_loss_history=val_loss_history,
    train_mae_history=train_mae_history,
    val_mae_history=val_mae_history,
    train_rmse_history=train_rmse_history,
    val_rmse_history=val_rmse_history
)

def fit(self, X: np.ndarray, y: np.ndarray, X_val: np.ndarray = None,
y_val: np.ndarray = None):
    if X.ndim == 1:
        X = X.reshape(-1, 1)

    if X_val is not None and X_val.ndim == 1:
        X_val = X_val.reshape(-1, 1)

```

```

self.gradient_descent(X, y, X_val, y_val)
self.is_fitted = True

if self.verbose:
    final_loss = self.compute_loss(y, self.predict(X))
    print(f"Training completed. Final loss: {final_loss:.6f}")

def predict(self, X: np.ndarray) -> np.ndarray:
    if not self.is_fitted:
        return X

    if X.ndim == 1:
        X = X.reshape(-1, 1)

    y_pred = np.dot(X, self.weights)
    if self.fit_intercept:
        y_pred += self.bias

    return y_pred

def score(self, X: np.ndarray, y: np.ndarray) -> float:
    y_pred = self.predict(X)
    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    return 1 - (ss_res / ss_tot)

def plot_metrics(self):
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    # MSE
    axes[0].plot(self.history.train_loss_history, label='Train')
    if self.history.val_loss_history:
        axes[0].plot(self.history.val_loss_history, label='Validation')
    axes[0].set_title('MSE during Training')
    axes[0].set_yscale('log')
    axes[0].legend()
    axes[0].grid(True)

    # MAE
    axes[1].plot(self.history.train_mae_history, label='Train')
    if self.history.val_mae_history:
        axes[1].plot(self.history.val_mae_history, label='Validation')
    axes[1].set_title('MAE during Training')
    axes[1].set_yscale('log')
    axes[1].legend()
    axes[1].grid(True)

```

```
# RMSE
axes[2].plot(self.history.train_rmse_history, label='Train')
if self.history.val_rmse_history:
    axes[2].plot(self.history.val_rmse_history, label='Validation')
axes[2].set_title('RMSE during Training')
axes[2].set_yscale('log')
axes[2].legend()
axes[2].grid(True)

plt.tight_layout()
plt.show()
```

Листинг 2 - реализация логистической регрессии

```
@dataclass
class TrainingHistoryLogistic:
    losses: list[float]
    accuracies: list[float]
    f1_scores: list[float]
    val_losses: list[float]
    val_accuracies: list[float]
    val_f1_scores: list[float]
    weights_history: list[np.ndarray]
    bias_history: list[float]

class LogisticRegression:
    def __init__(self, learning_rate: float = 0.01, n_iterations: int =
1000, tolerance: float = 1e-6, fit_intercept: bool = True, verbose: bool
= False):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.tolerance = tolerance
        self.fit_intercept = fit_intercept
        self.verbose = verbose
        self.weights = None
        self.bias = 0.0 if fit_intercept else None
        self.is_fitted = False
        self.history = None

    @staticmethod
    def sigmoid(z: np.ndarray) -> np.ndarray:
        return 1 / (1 + np.exp(-z))

    def compute_loss(self, y_true: np.ndarray, y_pred: np.ndarray) ->
float:
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1
- y_pred))

    def gradient_descent(self, X: np.ndarray, y: np.ndarray, X_val:
np.ndarray = None, y_val: np.ndarray = None):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        if self.fit_intercept:
            self.bias = 0.0
```

```

losses, accuracies, f1_scores = [], [], []
val_losses, val_accuracies, val_f1_scores = [], [], []
weights_history, bias_history = [], []

for iteration in range(self.n_iterations):
    linear_model = np.dot(X, self.weights)
    if self.fit_intercept:
        linear_model += self.bias
    y_pred_prob = self.sigmoid(linear_model)
    y_pred = (y_pred_prob >= 0.5).astype(int)

    # Loss & metrics
    loss = self.compute_loss(y, y_pred_prob)
    acc = accuracy_score(y, y_pred)
    f1 = f1_score(y, y_pred)

    losses.append(loss)
    accuracies.append(acc)
    f1_scores.append(f1)
    weights_history.append(self.weights.copy())
    if self.fit_intercept:
        bias_history.append(self.bias)

    # Validation metrics
    if X_val is not None and y_val is not None:
        val_prob = self.sigmoid(np.dot(X_val, self.weights) +
(self.bias if self.fit_intercept else 0))
        val_pred = (val_prob >= 0.5).astype(int)
        val_loss = self.compute_loss(y_val, val_prob)
        val_acc = accuracy_score(y_val, val_pred)
        val_f1 = f1_score(y_val, val_pred)

        val_losses.append(val_loss)
        val_accuracies.append(val_acc)
        val_f1_scores.append(val_f1)

    # Gradients
    error = y_pred_prob - y
    dw = (1 / n_samples) * np.dot(X.T, error)
    self.weights -= self.learning_rate * dw
    if self.fit_intercept:
        db = (1 / n_samples) * np.sum(error)
        self.bias -= self.learning_rate * db

    if iteration > 0 and abs(losses[-2] - losses[-1]) <
self.tolerance:

```

```

        if self.verbose:
            print(f"Converged at iteration {iteration}")
            break

        if self.verbose and iteration % 100 == 0:
            msg = f"Iteration {iteration}, Train Loss: {loss:.6f},
Acc: {acc:.3f}, F1: {f1:.3f}"
            if X_val is not None:
                msg += f", Val Loss: {val_loss:.6f}, Val Acc:
{val_acc:.3f}, Val F1: {val_f1:.3f}"
            print(msg)

    self.history = TrainingHistoryLogistic(
        losses=losses,
        accuracies=accuracies,
        f1_scores=f1_scores,
        val_losses=val_losses,
        val_accuracies=val_accuracies,
        val_f1_scores=val_f1_scores,
        weights_history=weights_history,
        bias_history=bias_history
    )

    def fit(self, X: np.ndarray, y: np.ndarray, X_val: np.ndarray = None,
y_val: np.ndarray = None):
        if X.ndim == 1:
            X = X.reshape(-1, 1)
        if X_val is not None and X_val.ndim == 1:
            X_val = X_val.reshape(-1, 1)

        self.gradient_descent(X, y, X_val, y_val)
        self.is_fitted = True

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        if not self.is_fitted:
            raise ValueError("Model is not fitted yet!")

        if X.ndim == 1:
            X = X.reshape(-1, 1)
        linear_model = np.dot(X, self.weights)
        if self.fit_intercept:
            linear_model += self.bias
        return self.sigmoid(linear_model)

    def predict(self, X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
        proba = self.predict_proba(X)

```

```

        return (proba >= threshold).astype(int)

def score(self, X: np.ndarray, y: np.ndarray) -> float:
    y_pred = self.predict(X)
    return np.mean(y_pred == y)

def plot_metrics(self):
    fig, axes = plt.subplots(1, 3, figsize=(18,5))

    # Loss
    axes[0].plot(self.history.losses, label='Train')
    if self.history.val_losses:
        axes[0].plot(self.history.val_losses, label='Validation')
    axes[0].set_title("Loss")
    axes[0].legend()
    axes[0].grid(True)

    # Accuracy
    axes[1].plot(self.history accuracies, label='Train')
    if self.history.val accuracies:
        axes[1].plot(self.history.val accuracies, label='Validation')
    axes[1].set_title("Accuracy")
    axes[1].legend()
    axes[1].grid(True)

    # F1 Score
    axes[2].plot(self.history.f1_scores, label='Train')
    if self.history.val_f1_scores:
        axes[2].plot(self.history.val_f1_scores, label='Validation')
    axes[2].set_title("F1 Score")
    axes[2].legend()
    axes[2].grid(True)

    plt.tight_layout()
    plt.show()

```

Листинг 3 - применение линейной регрессии

```
target_column = 'Median_House_Value'
X_cleaned = californication.drop(target_column, axis=1).values
y_cleaned = californication[target_column].values

separator = DataSeparator(test_size=0.15, val_size=0.15, shuffle=True,
seed=42)
data_splits = separator.separate(X_cleaned, y_cleaned)

scaler = StandardScaler()
scaler.fit(data_splits.X_train)

data_splits.X_train = scaler.transform(data_splits.X_train)
data_splits.X_val = scaler.transform(data_splits.X_val)
data_splits.X_test = scaler.transform(data_splits.X_test)

model = LinearRegression(learning_rate=0.01, n_iterations=1000)

model.fit(data_splits.X_train, data_splits.y_train, data_splits.X_val,
data_splits.y_val)

predictions = model.predict(data_splits.X_test)

r2_test = model.score(data_splits.X_test, data_splits.y_test)

model.plot_metrics()

print("Размеры выборок:")
print(f"Train: {data_splits.X_train.shape[0]} samples")
print(f"R^2 на тестовой выборке: {r2_test:.4f}")
print(f"Validation: {data_splits.X_val.shape[0]} samples")
print(f"Test: {data_splits.X_test.shape[0]} samples")
print(f"Features: {data_splits.X_train.shape[1]} features")
```

Листинг 4 - применение логистической регрессии

```
target_column = 'Survived'
X_cleaned = titanic.drop(target_column, axis=1).values
y_cleaned = titanic[target_column].values

separator = DataSeparator(test_size=0.15, val_size=0.15, shuffle=True,
seed=42)
data_splits = separator.separate(X_cleaned, y_cleaned)

scaler = StandardScaler()
scaler.fit(data_splits.X_train)

data_splits.X_train = scaler.transform(data_splits.X_train)
data_splits.X_val = scaler.transform(data_splits.X_val)
data_splits.X_test = scaler.transform(data_splits.X_test)

model = LogisticRegression(learning_rate=0.1, n_iterations=10000)

model.fit(data_splits.X_train, data_splits.y_train, data_splits.X_val,
data_splits.y_val)

predictions = model.predict(data_splits.X_test)

model.plot_metrics()

print("Размеры выборок:")
print(f"Train: {data_splits.X_train.shape[0]} samples")
print(f"Validation: {data_splits.X_val.shape[0]} samples")
print(f"Test: {data_splits.X_test.shape[0]} samples")
print(f"Features: {data_splits.X_train.shape[1]} features")
```

Оценки моделей

Далее на листингах 5-6 представлены результаты работы моделей:
Листинг 5 - результат работы линейной регрессии

```
Training completed. Final loss: 5469819243.300241, final MAE:
54931.848777, final RMSE: 73958.226340
R^2 на тестовой выборке: 0.5934
R^2 на обучающие выборке: 0.5885
```

Листинг 6 - результат работы логистической регрессии

```
Accuracy на обучающей выборке: 0.8041733547351525
F1 на обучающей выборке: 0.7479338842975206
Accuracy на тестовой выборке: 0.7518796992481203
F1 на тестовой выборке: 0.6373626373626373
```

Далее на рисунках 1-2 представлены графики изменения ошибок и метрик

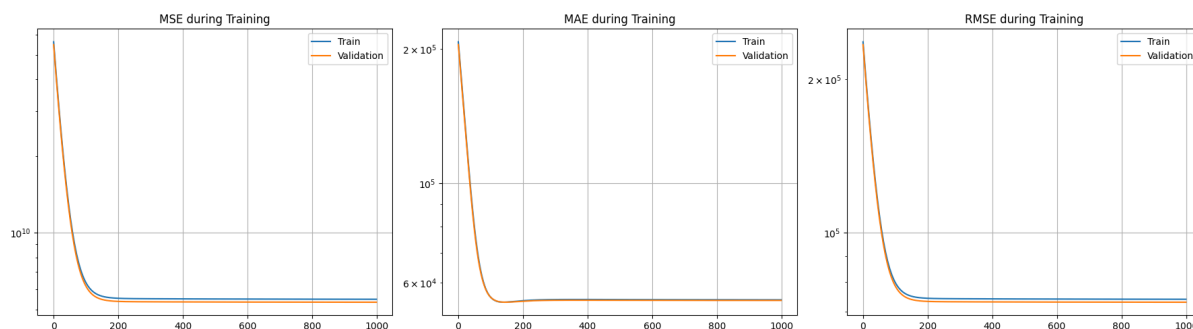


Рисунок 1. Графики изменения MSE, MAE и RMSE

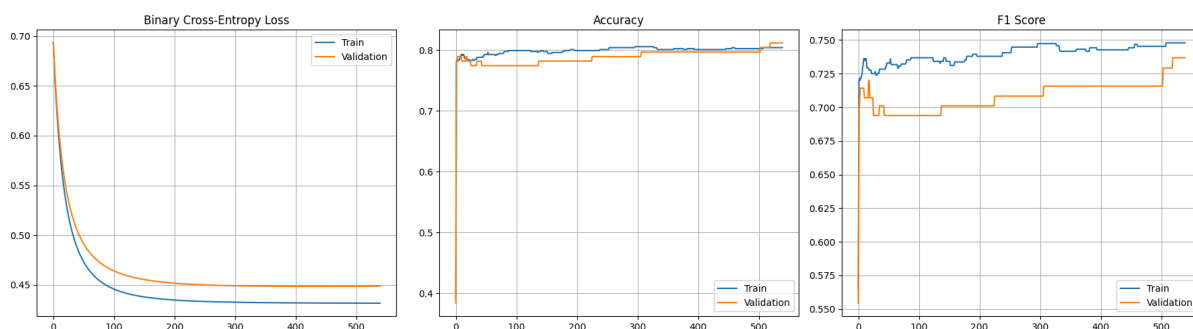


Рисунок 2. Графики изменения Loss, Accuracy и F1

Вывод

В процессе выполнения лабораторной работы я познакомилась с логистическим и линейным алгоритмами обучения, опробовала их на реальных данных и пронаблюдала прогресс обучения за счёт снятия метрик.